Home assignment #5

Home assignment 5: Планирование пути (Path Planning)

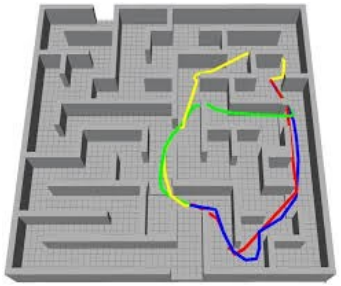
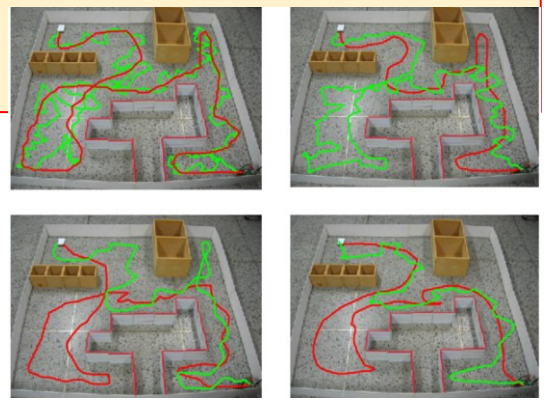
Задача: Реализовать конвейер поиска пути в 2D пространстве, сравнив классический поиск на сетке (A*) с алгоритмами на основе случайного сэмплирования (RRT/RRT*), и выполнить постобработку траектории.

Данные: Среды для планирования (Occupancy Grids):

- **Собственные карты:** Ч/Б изображения (PNG, ~200x200), созданные в графическом редакторе (черный — стена, белый — проход).
- **MovingAI Benchmark:** Профессиональные карты из игр (StarCraft, WarCraft III) или реальных городов.
- **PythonRobotics:** спользование встроенных в библиотеку генераторов карт.

Реализация в Jupyter/Colab (на базе Matplotlib и PythonRobotics):

1. **Базовое планирование (A* vs RRT):** Реализуйте два принципиально разных подхода. A* как эталон оптимальности на графе и RRT как стандарт для систем с высокой степенью свободы.
2. **Улучшенное планирование (RRT & Smoothing):**
   - Реализуйте **RRT*** — модификацию, которая перестраивает дерево для поиска кратчайшего пути.
   - Добавьте **Path Smoothing**: после нахождения пути прогоните его через алгоритм оптимизации (градиентное сглаживание или кривые Безье), чтобы избавиться от «ломаных» линий.
3. **Анализ и Визуализация:**
   - **Сравнение**: Проведите замеры по трем метрикам: длина пути, время вычислений и количество посещенных узлов/точек.
   - **Визуализация**: С помощью Matplotlib отрисуйте карту, процесс расширения дерева/сетки и итоговый сглаженный путь.
   - **Выводы**: Отразите, как разрешение карты влияет на скорость A* и почему RRT может испытывать трудности в узких коридорах.
4. **Результат:** Загрузите в Github ссылку на Colab, скриншоты сравнения траекторий («ломаный» путь vs «сглаженный») и небольшую анимацию (GIF/MP4) процесса поиска пути.

Где запускать и что использовать:

- **Библиотеки**: matplotlib (визуализация), opencv-python (загрузка карт), numpy.
- **NetworkX**: Рекомендуется, если планируете реализовывать алгоритмы на графах, а не на чистых матрицах.
- **Справочник**: Репозиторий AtsushiSakai/PythonRobotics как источник формул и логики.